

七. 项目描述八股文

7.2 FTP项目

7.2.1 简单说下你的项目

FTP命令是存在已久的文件传输工具，我学习Linux系统高级编程的时候，涉及文件，进程，线程，多线程，网络等知识点的开发，

根据已有的FTP命令的使用流程，自己开发了FTP服务端和客户端的代码，

主要功能是服务端支持多客户端接入并响应客户端发来的命令请求，实现服务端向客户端返回目录信息，文件信息数据，

并根据客户端需求，传送客户端要求的文件。

7.2.2 服务端设计方式

服务端的框架主要支持多路连接，在正常的socket服务端程序内加入线程管理，每当有新客户接入的时候就创建一个线程对接一个客户端。

那么系统就存在多个客户端对同一个目录或者文件的访问请求，使用线程同步方式来解决临界资源的访问问题。

（面试官如果打断，问线程同步，那么这块内容去Linux系统编程那块复习下）

因为多客户端连接，那么就要处理socket资源的利用效率，使用了select和poll机制来实现I/O多路复用，

这样服务端可以同时监控多个套接字，提高资源利用率和响应速度，允许非阻塞地处理多个客户端连接，从而增强程序的可伸缩性和效率。

（这里是引导面试官问select和poll，也到对应的Linux系统编程的章节去复习）

7.2.3 客户端的设计方式

客户端的设计相对于服务端就容易多了，主要是一个socket连接流程，当连接上服务器的时候就获取用户的键盘输入，然后处理

服务端返回的数据流，并展示给客户，更多是字符串的处理，比如字符串检测，字符串分割等。

7.2.4 支持断点续传吗，你的方案是什么-会问

实现断点续传功能，通常需要在发送端和接收端都记录文件的某个特定位置（比如已经成功传输的最后一个字节的位置），

这可以通过在发送端和接收端维护文件的校验和（如MD5）或使用文件偏移量来实现，确保数据的完整性和准确性。

并在下一次传输时从这个位置开始继续传输。以下是一个简单的示例，展示了如何使用socket编程实现断点续传的基本逻辑：

（md5: 有点像文件的身份证，一串数据，每个文件唯一）

以下是一个简单的示例，展示了如何使用socket编程实现断点续传的基本逻辑：

服务端代码（发送端）

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define PORT 8080
#define BUFFER_SIZE 1024

// 函数用于发送文件内容
int send_file(int client_socket, char *filename, long start, long end) {
    FILE *file = fopen(filename, "rb");
    if (!file) {
        perror("File opening failed");
        return -1;
    }

    fseek(file, start, SEEK_SET); // 定位到断网的位置

    char buffer[BUFFER_SIZE];
    int bytes_read;
    long total_sent = 0;

    while ((bytes_read = fread(buffer, 1, sizeof(buffer), file)) > 0) {
        if (total_sent + bytes_read > end - start + 1) {
            bytes_read = end - start + 1 - total_sent;
        }
        total_sent += bytes_read;

        int bytes_sent = send(client_socket, buffer, bytes_read, 0);
        if (bytes_sent < 0) {
            perror("Send failed");
            fclose(file);
            return -1;
        }
        if (bytes_sent != bytes_read) {
            // Handle partial send if needed
        }
        if (total_sent >= end - start + 1) {
            break;
        }
    }

    fclose(file);
    return 0;
}

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 创建套接字文件描述符
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
    }

```

```

        exit(EXIT_FAILURE);
    }

    // 绑定套接字到端口
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // 监听套接字
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Listening on port %d...\n", PORT);

    if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen)) < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    // 接收客户端发送的断点位置
    long start_position;
    recv(new_socket, &start_position, sizeof(start_position), 0);

    // 发送文件
    send_file(new_socket, "example.txt", start_position, sizeof(buffer));

    close(new_socket);
    close(server_fd);
    return 0;
}

```

客户端代码 (接收端)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

// 函数用于接收文件内容
int receive_file(int server_socket, char *filename, long start) {
    FILE *file = fopen(filename, "ab"); // Append mode

```

```

    if (!file) {
        perror("File opening failed");
        return -1;
    }

    char buffer[BUFFER_SIZE];
    int bytes_received;

    long total_received = 0;

    while ((bytes_received = recv(server_socket, buffer, sizeof(buffer), 0)) > 0)
    {
        fwrite(buffer, 1, bytes_received, file);
        total_received += bytes_received;
    }

    fclose(file);
    return 0;
}

int main(int argc, char const *argv[]) {
    int sock = 0;
    struct sockaddr_in serv_addr;
    long start_position = 0; // 假设这是保存的断点位置

    if (argc != 2) {
        printf("Usage: %s <server_ip>\n", argv[0]);
        return 1;
    }

    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, argv[1], &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // 发送断点位置到服务器
    send(sock, &start_position, sizeof(start_position), 0);

    // 接收文件
    receive_file(sock, "example.txt", start_position);

    close(sock);
    return 0;
}

```

```
}
```

这个示例中，客户端和服务端都维护了一个断点位置 `start_position`。

客户端在连接服务端后，会发送这个位置给服务端，

服务端根据这个位置来决定从文件的哪个位置开始发送数据。

这样，即使传输过程中断，下次连接时也能从上次中断的地方继续传输。

7.2.5 其他技术问题

其他的具体实现细节，比如命令的处理，数据返回等，根据自己做的项目来回答。